Actividad 2.4

Universidad Politécnica de Atlautla.

Ing. En tecnologías de la Información e Innovación Digital

Fundamentos de Inteligencia Artificial.

2.4 Algoritmos de Busqueda ciega BFS y DFS.

Docente: Mtro. Javier García Lara

Alumno: Fabian Andrade Hernandez

Grupo: 172

Semana: 25 oct.

Secciones 4

Algoritmos de búsqueda ciega: BFS y DFS

Código y comparativa aplicada a la resolución de un laberinto

Nombre: Fabian Andrade Hernandez

Materia: Fundamentos de la Inteligencia Artificial.

Fecha: 25/10/2026

1. Introducción y objetivo

Los algoritmos de búsqueda ciega (no informada) exploran el espacio de estados sin utilizar ningún conocimiento sobre qué tan cerca está una configuración de la meta. En esta actividad se implementan en Python la búsqueda en anchura (BFS, Breadth-First Search) y la búsqueda en profundidad (DFS, Depth-First Search) para resolver un laberinto, con el fin de documentar su funcionamiento, analizar su complejidad y compararlos en términos de tiempo, nodos expandidos y calidad de la solución.

2. Descripción del problema y representación del grafo

El laberinto se genera de forma procedural con el algoritmo de backtracking recursivo (implementado de forma iterativa) y una semilla fija (seed = 7) para que los resultados sean reproducibles. Después se derriban 40 muros al azar para introducir ciclos; sin ellos el laberinto sería "perfecto" (con un único camino entre dos celdas) y BFS y DFS devolverían la misma ruta. Dimensiones: 21 × 41 celdas, con 414 celdas libres.

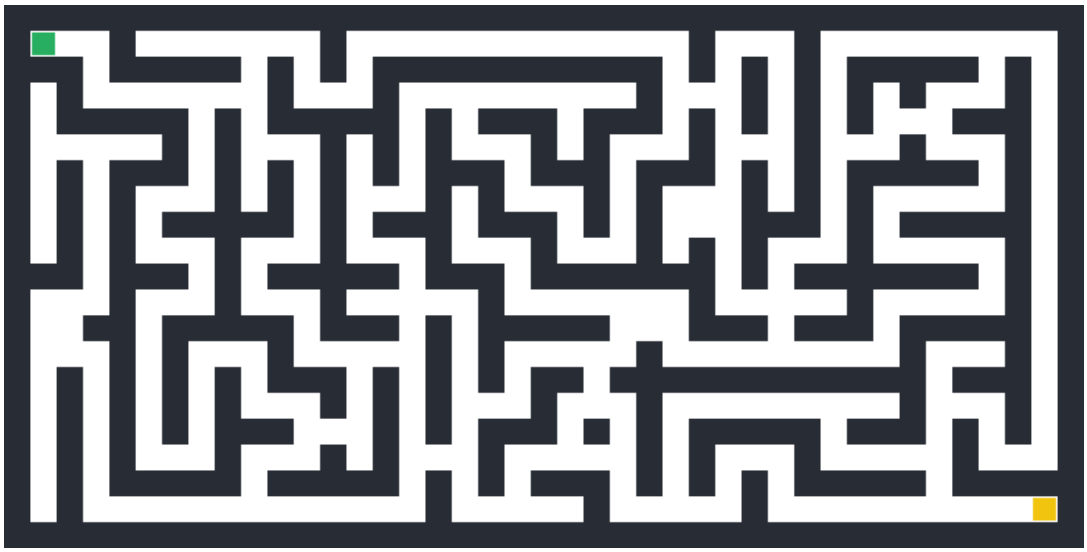


Figura 1. Laberinto generado. Verde: inicio (S). Amarillo: meta (G).

Representación. El laberinto se almacena como una matriz de enteros (0 = pasillo, 1 = muro). A partir de ella se define un grafo implícito:

- **Nodos (estados):** cada celda libre (fila, columna). Total: V = 414.
- **Aristas (acciones):** movimientos a las cuatro celdas vecinas (arriba, derecha, abajo, izquierda) siempre que sean pasillo. Costo uniforme igual a 1.
- **Estado inicial:** (1, 1). **Meta:** (19, 39).
- **Ventaja de la representación implícita:** los vecinos se calculan al vuelo con una función generadora, por lo que no es necesario construir ni almacenar listas de adyacencia.

3. Implementación

Ambos algoritmos comparten la misma estructura: una frontera de nodos por expandir, un diccionario de padres (que sirve a la vez como conjunto de visitados y para reconstruir el camino) y contadores de nodos expandidos y de tamaño máximo de la frontera. La única diferencia es la estructura de datos de la frontera:

- **BFS:** cola FIFO (collections.deque). Expande por niveles, de modo que la primera vez que llega a la meta lo hace por el camino con menos pasos.
- **DFS:** pila LIFO (lista de Python). Avanza por una rama hasta agotarla y luego retrocede (backtracking).

El orden de expansión de los vecinos es fijo (N, E, S, O) para que la comparación sea justa. El código completo se incluye en el Anexo y en el archivo laberinto_bfs_dfs.py.

4. Análisis de complejidad

Sea V el número de nodos, E el de aristas, b el factor de ramificación (máximo 4 en la cuadrícula), d la profundidad de la solución menos profunda y m la profundidad máxima del espacio de búsqueda.

Propiedad	BFS	DFS
Complejidad temporal	$O(V + E) \approx O(b^d)$	$O(V + E) \approx O(b^m)$
Complejidad espacial	$O(V) \approx O(b^d)$: la frontera crece con el ancho del grafo	O(V) con conjunto de visitados; O(b·m) en árbol sin visitados
Completitud	Sí (espacio finito)	Sí, si se controlan los estados repetidos
Optimalidad	Sí, con costo uniforme por paso	No: devuelve el primer camino encontrado
Frontera	Cola (FIFO)	Pila (LIFO)

En la cuadrícula, V ≤ filas × columnas y E ≤ 2V, por lo que en la práctica ambos algoritmos son lineales en el tamaño del laberinto. La diferencia relevante está en el orden de exploración, que afecta cuántos nodos se expanden antes de llegar a la meta y la calidad del camino obtenido.

5. Resultados experimentales

Cada algoritmo se ejecutó 500 veces sobre el mismo laberinto y se reporta el tiempo promedio por ejecución (time.perf_counter). Los tiempos absolutos dependen del equipo; las métricas de nodos expandidos y longitud del camino son deterministas.

Algoritmo	Longitud del camino	Nodos expandidos	Frontera máxima	Tiempo (ms)
BFS	68	391	12	0.522
DFS	88	132	9	0.130

Tabla 1. Comparativa de BFS y DFS en el laberinto de 21×41.

5.1 Camino solución de BFS

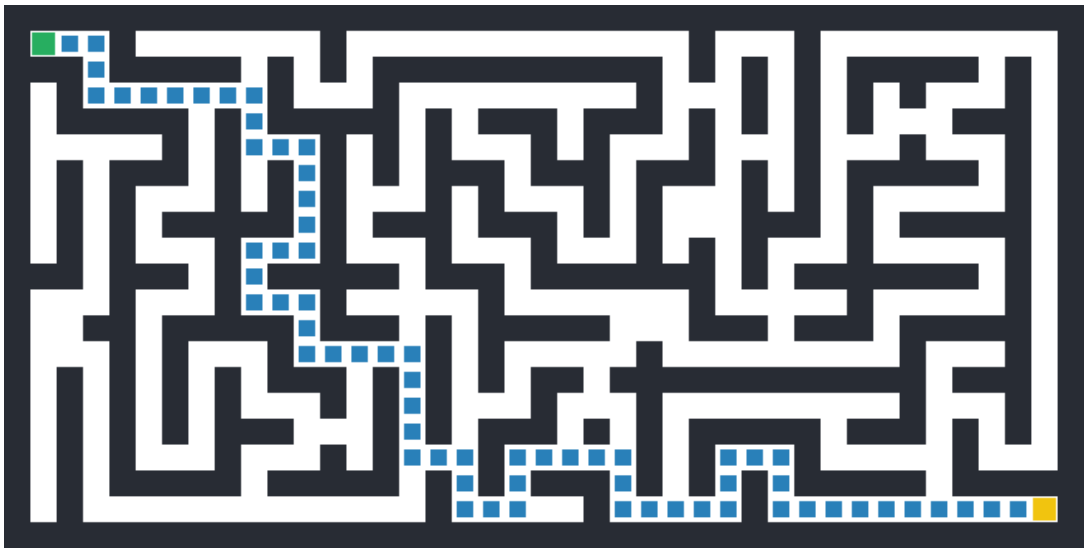


Figura 2. Camino encontrado por BFS (azul): 68 pasos, el mínimo posible.

5.2 Camino solución de DFS

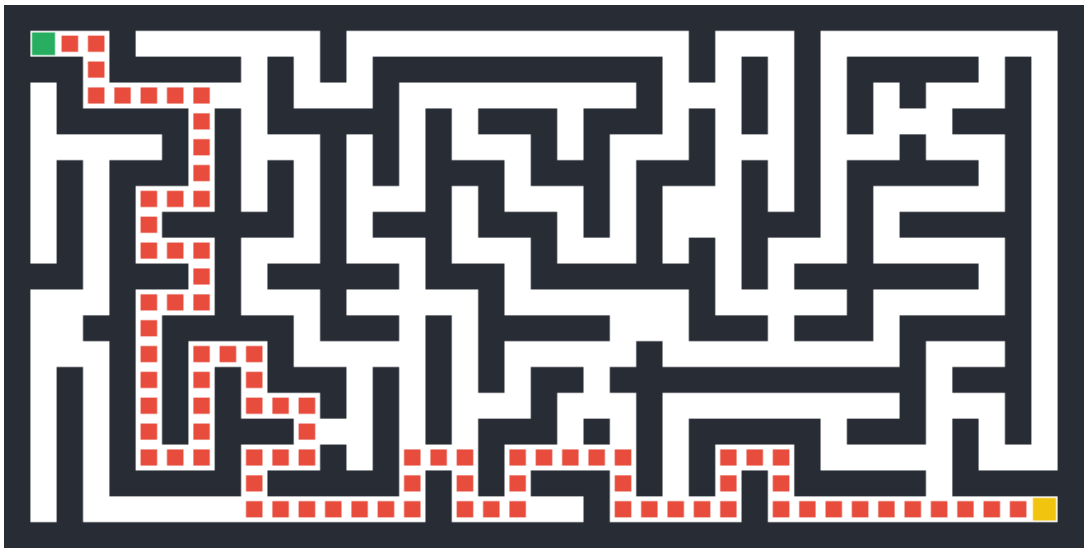


Figura 3. Camino encontrado por DFS (rojo): 88 pasos, con rodeos innecesarios.

6. Comparación y discusión

- **Calidad de la solución.** BFS garantizó el camino más corto (68 pasos). DFS devolvió una solución válida pero 29% más larga (88 pasos), pues se compromete con la primera rama que explora y no revisa alternativas más cortas.
- **Nodos expandidos y tiempo.** En este laberinto DFS expandió 66% menos nodos (132 frente a 391) y fue más rápido. BFS debe explorar todos los nodos a distancia menor que la de la meta antes de alcanzarla, lo que en este caso cubre casi todo el laberinto.
- **Memoria.** La frontera máxima fue similar (12 vs 9) porque el laberinto es estrecho. En espacios con mucha ramificación, la frontera de BFS crece exponencialmente, mientras que la de DFS crece de forma lineal con la profundidad.
- **Sensibilidad.** La ventaja de DFS en nodos expandidos depende de la posición de la meta y del orden de los vecinos: en el peor caso también puede recorrer casi todo el grafo. BFS, en cambio, tiene un comportamiento más predecible.

7. Conclusión: idoneidad según el problema

Ninguno de los dos algoritmos es mejor en todos los casos; la elección depende de lo que se necesite del problema:

- **BFS es adecuado** cuando se requiere el camino más corto (costo uniforme), cuando la meta está cerca del inicio o cuando el espacio es pequeño o con poca ramificación, y se puede pagar el costo de memoria.
- **DFS es adecuado** cuando basta con encontrar cualquier solución, cuando el espacio de búsqueda es muy profundo o con mucha ramificación y la memoria es limitada, o cuando se necesita explorar todo el grafo (detección de ciclos, conectividad, generación de laberintos).

Para el problema del laberinto, donde normalmente interesa la ruta más corta, BFS es la opción más apropiada: asegura optimalidad a un costo computacional moderado (0.52 ms). DFS es útil como alternativa económica en memoria, pero sacrifica la calidad de la ruta. Para problemas con costos desiguales o grafos muy grandes, sería preferible pasar a métodos informados como A*.

8. Referencias

- Bohm, cap. 3: Búsqueda no informada (material del curso).
- Russell, S. y Norvig, P. Artificial Intelligence: A Modern Approach. Cap. 3: Solving Problems by Searching.
- Promt; Anthropic. (2026). Claude 5.5. https://claude.ia/ (Prompt: Resolución de actividad y programación de código)

Anexo. Código de BFS y DFS

Funciones principales (el script completo con el generador del laberinto está en laberinto_bfs_dfs.py).

```
def bfs(m, inicio, meta):
    frontera = deque([inicio])
    padres = {inicio: None}
    expandidos, max_front = 0, 1
    while frontera:
        max_front = max(max_front, len(frontera))
        n = frontera.popleft()          # FIFO
        expandidos += 1
        if n == meta:
            return reconstruir(padres, meta), expandidos, max_front
        for v in vecinos(m, n):
            if v not in padres:
                padres[v] = n
                frontera.append(v)
    return None, expandidos, max_front

def dfs(m, inicio, meta):
    frontera = [inicio]
    padres = {inicio: None}
    visitados = set()
    expandidos, max_front = 0, 1
    while frontera:
        max_front = max(max_front, len(frontera))
        n = frontera.pop()              # LIFO
        if n in visitados:
            continue
        visitados.add(n)
        expandidos += 1
        if n == meta:
            return reconstruir(padres, meta), expandidos, max_front
        for v in vecinos(m, n):
            if v not in visitados:
                padres[v] = n           # último padre que lo apila
                frontera.append(v)
    return None, expandidos, max_front
```